Note Feller
Design and Implementation

Jacob Burtenshaw
Chris Kane-Pardy
Niklas Anderson
Sajida Sayyad

June 10, 2026

Contents

1. Overview

Note Feller is a rhythm-based music game implemented as a System-on-Chip (SoC) design on the Digilent Nexys A7 (Xilinx Artix-7) FPGA platform. The system scrolls note targets down a VGA monitor while the player matches note timing using the onboard pushbuttons or a USB keyboard. Correct, well-timed inputs raise the score and extend a combo multiplier; mistimed or missed notes reset the combo.

The project follows a hardware/software co-design split:

- **Custom SystemVerilog peripherals** handle the real-time work — VGA scan-out, audio synthesis, and input sampling — exposing their state to the CPU through memory-mapped Wishbone registers.
- **Embedded C firmware** running on the VeeR EL2 RISC-V core (`NoteFeller-App/`) owns all gameplay logic: note sequencing, movement, hit detection, scoring, audio cues, and menu/state control.

We proceeded with development incrementally, validating each subsystem independently (VGA and input first, then audio and gameplay integration) before full system bring-up. The codebase started from the Project 2 VGA peripheral and was extended with new audio, input, and USB address space.

2. System Architecture

2.1 Wishbone interconnect and address space

The peripherals attach to the existing VeeRwolf Wishbone I/O bus. Adding the new hardware required extending `wb_intercon.v` / `wb_intercon.vh` with slave slots for audio, USB, and a second GPIO bank, plus an enlarged region for VGA. The AXI-to-Wishbone bridge address window in `axi_intercon.sv` was widened from `0x80004000` to `0x80010000` to cover the new slaves.

Sizing rationale: the input-controller bank reuses the compact 64-byte `gpio2` slot, while audio, VGA, and USB each receive a full 4 KiB page. The generous per-page allocation leaves room for register-map growth — the VGA sprite table alone consumes 64 sprites × 2 words = 512 bytes.

2.2 Memory map

All peripherals live in the I/O region based at `0x80000000`.

Peripheral	Base address	Size	Module
Boot ROM	`0x80000000`	4 KiB	`wb_mem_wrapper`
System controller	`0x80001000`	64 B	`veerwolf_syscon` (7-seg)
SPI flash	`0x80001040`	64 B	`simple_spi_top`
SPI accelerometer	`0x80001100`	64 B	`simple_spi_top`
PIT/timer (PTC)	`0x80001200`	64 B	`ptc_top`
GPIO	`0x80001400`	64 B	`gpio_top`
Input controller	`0x80001500`	64 B	`wb_input_controller`
UART	`0x80002000`	4 KiB	`uart_top`
VGA	`0x80003000`	4 KiB	`wb_vga`
Audio	`0x80004000`	4 KiB	`wb_audio`

Peripheral	Base address	Size	Module
USB/PS-2	`0x80008000`	4 KiB	reserved

Bold rows are subsystems built or extended for Note Feller. Each custom slave implements the same minimal Wishbone handshake: a registered one-cycle `wb_ack_o` asserted when `wb_cyc_i & wb_stb_i` is seen, with `wb_err_o` and `wb_rty_o` tied low.

3. Audio Subsystem

3.1 `wb_audio` — direct digital synthesis with delta-sigma output

The Nexys A7 exposes only a single-bit PWM audio pin (`aud_pwm`) plus a shutdown/mute pin (`aud_sd`). The audio peripheral therefore synthesizes tones digitally and reduces them to a 1-bit stream.

Phase accumulators (DDS). Each voice owns a 24-bit phase accumulator that is incremented every clock by a per-note phase-increment constant drawn from a note look-up table (`NOTE_LUT`). A larger increment wraps the accumulator sooner, producing a higher frequency. The accumulator's most-significant bit (bit 23) is taken as a square wave at the target pitch.

Polyphony. To sound chords, the single-voice design was widened to eight parallel voices (notes C4–C5). The phase accumulators, square-wave MSBs, per-voice volume/amplitude, and sample values are all 8-element arrays. Each voice's square wave is scaled to a signed amplitude (`±volume`), gated by its on/off bit, and the eight contributions are summed.

Delta-sigma 1-bit DAC. The summed sample is right-shifted (averaged), biased to a positive value, and fed into a delta-sigma modulator accumulator. The carry/MSB of that accumulator drives `aud_pwm` directly, so the time- average of the PWM stream reconstructs the mixed analog waveform while pushing quantization noise out of the audible band.

3.2 Register map (`wb_audio`, base `0x80004000`)

Offset	Name	Field encoding
`0x00`	`AUDIO_CTRL`	`[0]` amp enable (drives `aud_sd`); `[7:4]` reserved master gain
`0x04`	`AUDIO_VOICES`	`[7:0]` per-voice on/off mask — bit i = voice i sounding
`0x08`	`AUDIO_VOL`	Eight packed 4-bit volumes: voice$i$ in bits `[i*4 +: 4]`

The `AUDIO_VOL` packing places C4 in `[3:0]`, D4 in `[7:4]`, … C5 in `[31:28]`, giving independent per-note balance within a chord.

3.3 Worked example: note frequency → phase increment

The output frequency of a phase accumulator of width N clocked at f_{clk} with increment P is:

$$f_{\text{out}} = P \cdot f_{\text{clk}} \,/\, 2^N$$

Solving for the increment stored in the LUT (here $N = 24$, $f_{\text{clk}} = 25$ MHz):

$$P = \text{round}(\ f_{\text{note}} \ \cdot \ 2^{24} \ / \ 25\ 000\ 000\ )$$

Example — A4 = 440 Hz:

$$P = \text{round}(440 \ \cdot \ 16\ 777\ 216 \ / \ 25\ 000\ 000) = \text{round}(295.28) = \mathbf{295 = 0x127}$$

Reconstructing the realized pitch: $295 \ \cdot \ 25$ MHz $/ \ 2^{24} =$ **439.6 Hz**, within 0.1 % of concert A. The full table the hardware ships with:

Voice	Note	Freq (Hz)	Computed P	LUT value
0	C4	261.63	175.6	`0x0000B0`
1	D4	293.66	197.1	`0x0000C5`
2	E4	329.63	221.2	`0x0000DD`
3	F4	349.23	234.4	`0x0000EA`
4	G4	392.00	263.1	`0x000107`
5	A4	440.00	295.3	`0x000127`
6	B4	493.88	331.5	`0x00014C`
7	C5	523.25	351.2	`0x00015F`

3.4 `audio.c` / `audio.h` driver

The software driver hides register packing behind a small voice-oriented API: `audio_init()` silences all voices, loads a uniform default volume, and enables the amplifier; `audio_set_voice(idx, on)` and `audio_set_voices(mask)` toggle notes; `audio_set_voice_volume()` performs a read-modify-write of the packed `AUDIO_VOL` register. Voice indices map directly to scale notes (`AUDIO_VOICE_C4` … `AUDIO_VOICE_C5`), so the game can play a lane's note with a single call.

4. Video Subsystem (VGA)

4.1 Display timing

A display-timing generator (`dtg.sv`) produces 640×480 @ 60 Hz sync signals from a 25 MHz pixel clock, emitting `pixel_row`, `pixel_column`, and `video_on`. The Wishbone clock doubles as the pixel clock, so VGA timing and register access share one domain.

4.2 Sprite engine and scan-line prefetch FSM

The design supports up to 64 sprites concurrently on the vga display. The CPU programs sprite registers with relevant information (type, color, position, ROM id) ahead of time. Then the hardware composites them live during scan-out.

While sprite information is stored in a sprite register, while the sprite bit-maps are stored in a ROM. During each row's horizontal blanking interval the sprite maps are read out in advance for the *next* scan line of every sprite that overlaps it. The BRAM read latency is one sycle so the fetch uses 65 of the ~160 available blanking cycles. A two-layer combinational compositor then paints each visible pixel: the background-color register forms the bottom layer, and sprites are applied in priority order (sprite 0 wins). A `0` bitmap bit is transparent; `1` bits paint the sprite's foreground color.

Each descriptor selects a **32×32** or **16×16** sprite, allowing both chunky note/chord circles and compact text glyphs from the same engine.

> **Resource note.** The original goal was a full 12-bpp 640×480 @ 60 Hz frame buffer in DDR2 with a DMA path, but the BRAM/DMA implementation could not be finished in the available time, so the sprite engine was adopted as a resource-efficient partial solution.

4.3 Sprite ROM

A unified Vivado Block-Memory ROM (32-bit × 4096, init from `guitarSprites.coe`) holds both worlds: IDs **0–94** are bigfont glyphs (ASCII 32–126, `sprite_id = ascii - 32`) and IDs **95–111** are the guitar/game sprites (chord circles, note circles, squiggles, borders). The ROM address is `{sprite_id[6:0], row_offset[4:0]}`. Unifying the text font and game art into one ROM is what let the score and menus move off the seven-segment display and onto the VGA screen.

4.4 Register map (`wb_vga`, base `0x80003000`)

Offset	Name	Field encoding
`0x000`	`REG_MODE`	`[0]` display mode (0 = graphics)
`0x014`	`REG_BG_COLOR`	`[11:0]` background `{R[11:8],G[7:4],B[3:0]}`
`0x080 + n·8`	`SPRITE_POS[n]`	`[9:0]` x position; `[19:10]` y position
`0x084 + n·8`	`SPRITE_CFG[n]`	`[31:25]` sprite id; `[15:4]` color RGB444; `[1]` type (0=32×32, 1=16×16); `[0]` visible

4.5 `vga_sprite.c` / `vga_sprite.h` driver

The driver presents a `Sprite` descriptor struct (slot, id, type, color, x, y) and packs it into the two hardware words. Helpers include `VGA_COLOR(r,g,b)` for 12-bit colors, named `SPRITE_FORM_*` constants for every ROM entry, `vga_set_sprite()`, `vga_clear_sprite()`, and `vga_set_bg_color()`. Higher-level game modules (keys, notes, menu, score) build entirely on this API.

5. Input / I-O Subsystem

5.1 `wb_input_controller`

The input controller converts asynchronous user inputs into a CPU-friendly, memory-mapped interface for gameplay. Rather than exposing raw button or keyboard signals directly to software, the peripheral presents both the current input state and a set of latched press events through Wishbone registers.

For onboard pushbuttons, the controller first synchronizes the asynchronous button signals into the system clock domain using a two-stage flip-flop synchronizer. The synchronized inputs are then compared against their previous sampled values to detect rising-edge transitions. When a new press is detected, the corresponding bit is latched in an edge-event register. These edge bits remain asserted until cleared by software, preventing short button presses from being missed even if the processor polls the peripheral at a slower rate than the hardware sampling frequency.

This distinction between **held state** and **new-press events** is critical for rhythm-game scoring. The current input state allows the software to determine whether a key is being held, while the edge register identifies the exact moment a new key press occurred. This prevents a held button from repeatedly generating hit events while still allowing the game to react immediately to new inputs.

The peripheral also supports multiple input sources through a selectable input mode register. During development, gameplay could be controlled either by the Nexys A7 pushbuttons or by a keyboard interface while presenting the same five-bit gameplay abstraction to software. Regardless of the physical source, the controller outputs a common set of lane signals corresponding to the four gameplay lanes and the Enter/start button.

5.2 Keyboard Input Processing

Keyboard input is represented internally using the same five-bit gameplay interface as the pushbuttons. The controller receives decoded keyboard scan-code events and translates them into lane states corresponding to the A, S, D, F, and Enter keys.

A dedicated receiver module captures serial keyboard data by synchronizing the incoming clock signal, detecting clock falling edges, and shifting the incoming 11-bit frame into a register. After a complete frame has been received, the module outputs the received scan code along with a one-cycle valid pulse. The controller then interprets keyboard make and break sequences to maintain the current key state.

When a key's make code is received, the corresponding gameplay input bit is asserted. When a break sequence (`0xF0`) is followed by a key's scan code, the associated gameplay bit is cleared. This allows keyboard inputs to behave identically to held pushbuttons from the software's perspective.

Current key mappings are:

Key	Gameplay Signal
A	Lane 0
S	Lane 1
D	Lane 2
F	Lane 3
Enter	Start / Menu

This translation layer allows the game software to remain completely agnostic to the underlying input device.

5.3 Register Map (`wb_input_controller`, base `0x80001500`)

Offset	Name	Field Encoding
`0x00`	`INPUT_STATUS`	`[4:0]` current held inputs (lanes 0–3 + Enter)
`0x04`	`INPUT_EDGE`	`[4:0]` latched rising-edge events; write 1s to clear
`0x08`	`INPUT_CTRL`	`[0]` clear all edge-event bits
`0x0C`	`INPUT_MODE`	`[0]` input source select (`0 = buttons`, `1 = keyboard`)

The separation between INPUT_STATUS and INPUT_EDGE allows software to distinguish between keys that are currently held and keys that were newly pressed since the last poll. This design simplifies gameplay logic while reducing the risk of missed timing events.

5.4 input_controller.c / input_controller.h Driver

The software driver provides a small abstraction layer over the hardware register interface. Functions such as input_get_status() and input_get_edges() expose the current held-input state and latched press events without requiring the game engine to interact with raw memory-mapped addresses.

The most commonly used interface is input_poll_new_presses(), which reads the edge-event register, clears the bits that were observed, and returns the resulting press mask. This operation effectively converts the hardware edge latches into a software event queue and serves as the primary hit-detection mechanism used by the game loop.

Additional helper functions allow software to clear selected edge bits, clear all pending events, and select the active input source. Because both pushbutton and keyboard inputs are reduced to the same five-bit gameplay representation in hardware, the remainder of the game engine can operate independently of the physical input device.

6. Seven-Segment Display (legacy score output)

Before the score was rendered on the VGA screen, it was shown on the board's eight-digit seven-segment display, driven through the System Controller. The seven_segment.c driver writes two registers: an enables register (0x80001038) and a packed-digit register (0x8000103C). sevenseg_display_score() converts the decimal score into packed BCD nibbles. The module is still updated in parallel with the VGA score for redundancy, but the VGA bigfont text (Section 4.3) is now the primary readout.

Offset	Name	Encoding
0x80001038	SEVENSEG_ENABLES	Per-digit enable (bit = 1 disables digit)
0x8000103C	SEVENSEG_DIGITS	Eight 4-bit hex nibbles, one per digit

7. Game Engine (NoteFeller-App)

7.1 Software Architecture

The Note Feller application is implemented as a collection of modular software components running on the VeeR EL2 processor. Rather than embedding gameplay behavior inside the hardware peripherals, the hardware is used only to provide deterministic services such as video generation, audio synthesis, and input capture. All gameplay decisions are made in software.

The game engine is organized into several cooperating modules:

Module	Responsibility
main.c	High-level game loop and state machine
input_controller.c	Hardware input abstraction

Module	Responsibility
`note.c`	Note spawning, movement, and hit detection
`key.c`	Lane target graphics and lane-state visualization
`score.c`	Score, combo, and multiplier management
`audio.c`	Audio peripheral control
`menu.c`	Start screen, HUD, and game-over display
`vga_sprite.c`	Low-level sprite interface

This separation allows gameplay features to be modified without requiring changes to the underlying hardware peripherals.

7.2 Main Loop and State Machine

The game operates as a finite-state machine with three primary states:

State	Purpose
`START`	Display title screen and wait for player input
`PLAYING`	Execute gameplay logic and update active notes
`END`	Display final score and wait for restart

The main loop repeatedly executes the update routine associated with the current state. State transitions occur only in response to player input or game-completion conditions.

At startup, all peripherals and gameplay modules are initialized before entering the start-screen state. Pressing Enter transitions into gameplay, while reaching the end of a run transitions to the game-over screen. Pressing Enter again resets all gameplay state and begins a new session.

This approach keeps gameplay behavior deterministic and avoids deeply nested control structures.

7.3 Lane Abstraction

The game is built around four independent gameplay lanes. Each lane is associated with:

- A dedicated input bit
- A unique screen position
- A unique display color
- A corresponding audio voice

This mapping allows the game engine to process all lanes using the same logic while maintaining independent visual and audio feedback.

Lane configuration information is stored in shared lookup tables, allowing gameplay routines to iterate over lanes rather than implementing separate logic for each key.

7.4 Note Lifecycle

Each active note progresses through a simple lifecycle:

1. **Spawned**

2. **Moving**
3. **Hittable**
4. **Hit or Missed**
5. **Removed**

Notes are stored in a fixed-size pool organized by lane. This eliminates dynamic memory allocation and guarantees bounded memory usage throughout gameplay.

When a note is activated, it is assigned a sprite register and begins moving downward at a fixed rate. As the note approaches the target region near the bottom of the screen, it becomes eligible for scoring. Once the note is either successfully hit or leaves the visible play area, it is removed and its sprite is cleared.

Because the note pool is statically allocated, no memory allocation or deallocation occurs during gameplay.

7.5 Note Movement and Timing

Note movement is implemented using software timing counters rather than hardware interrupts. Each active note maintains an internal tick counter that determines when its position should be updated.

When the counter exceeds a predefined threshold, the note advances by a fixed number of pixels and the associated sprite position is updated in hardware.

This approach provides deterministic movement behavior while avoiding the complexity of interrupt-driven animation systems. Difficulty can be adjusted by modifying the movement increment, update threshold, spawn rate, or note density.

7.6 Hit Detection

Hit detection is performed entirely in software using the edge-latched input events provided by the input controller.

When a new key press is detected, the game checks the corresponding lane for an active note currently inside the hit region. A successful match results in:

- Note removal
- Score increase
- Combo increment
- Audio feedback
- Visual lane feedback

If no eligible note is present when the key is pressed, the action is treated as a miss and the combo multiplier is reset.

Using edge-latched inputs prevents held keys from generating repeated scoring events and allows player timing to be evaluated based on discrete press events.

7.7 Scoring System

The scoring subsystem maintains three primary values:

- Score

- Combo count
- Score multiplier

Successful hits increase both score and combo count. The multiplier increases automatically as the combo grows, rewarding sustained accuracy. Missed notes or incorrect key presses reset the combo and return the multiplier to its minimum value.

The scoring module is responsible for updating both the VGA score display and the seven-segment display, providing a consistent interface for the rest of the game engine.

7.8 Visual Feedback

Visual feedback is used to communicate gameplay state to the player.

Each lane target changes appearance in response to successful hits or misses. Falling notes are represented as independent sprite objects whose positions are continuously updated throughout gameplay. Additional sprite resources are used for menus, score displays, and combo indicators.

Because all graphical elements are implemented using hardware sprites, the software only updates sprite descriptors rather than manipulating individual pixels. This significantly reduces processor workload while allowing smooth animation.

7.9 Resource Allocation

The game uses the 64 available hardware sprite registers to display notes, lane targets, score indicators, and menu graphics.

Sprite registers are statically assigned to gameplay objects wherever possible. Menu screens and gameplay screens reuse portions of the sprite address space because they are never displayed simultaneously.

This static allocation strategy simplifies sprite management and avoids runtime resource contention while ensuring all graphical objects have deterministic hardware resources available.

8. Key Design Decisions

Decision	Reason
64-sprite scan-line engine instead of a frame buffer	A 12-bpp 640×480 frame buffer exceeded available BRAM; the DDR2 + DMA path could not be finished in time.
Direct digital synthesis (phase accumulators) for tones	One adder per voice generates any pitch from a small LUT; trivially scales to polyphony.
Delta-sigma 1-bit DAC on `aud_pwm`	The board provides only a 1-bit PWM audio pin; noise-shaping yields acceptable tone quality.
Sum-and-bias voice mixing	Lets eight independent voices share the single PWM output as a chord.
Hardware edge detection + latched event register	Cleanly distinguishes a new press from a held key and decouples input timing from CPU poll rate.
Selectable pushbutton / keyboard input source	Either device can drive the same five gameplay signals, easing development and enabling the keyboard stretch goal.

Decision	Reason
Reuse the 64 B `gpio2` slot for input	Minimal interconnect change for a register set that needs little space.
4 KiB pages for audio/VGA/USB	Headroom for register-map growth (e.g., the 512 B sprite table) and future features.
Unified font + sprite ROM	One ROM serves both game art and text, allowing score/menus to render on VGA.
Per-lane hit-flag lock state	Prevents a held key from double-scoring; one press = one scoring event.
Sustain countdown for note-off	Decouples audio duration from key-hold time for more realistic sound.
`__TIME__`-seeded `srand`	Varies spawn patterns without depending on a cycle-counter CSR.

9. Summary

Note Feller demonstrates a complete hardware/software co-designed SoC on the Nexys A7. Three custom Wishbone peripherals carry the real-time load: a polyphonic DDS audio engine that mixes eight voices into a single delta-sigma PWM pin, a 64-sprite scan-line VGA engine that composites game art and text from a unified ROM without a frame buffer, and an input controller that adds edge detection and a selectable button/keyboard source on top of raw GPIO. A thin driver layer maps each peripheral to a clean C API, and the firmware game engine layers note spawning, fixed-speed movement, lane-locked hit detection, and a tiered combo-scoring system on top. The design's recurring theme is trading ideal-but-costly approaches (frame buffer, multi-bit DAC) for resource-efficient FPGA-friendly ones (sprite engine, sigma-delta), delivering a responsive, musical game within the platform's BRAM and I/O constraints.

10. LLM Acknowledgement

Portions of the software, hardware, and documentation were developed with the assistance of large language models. Generated content was used primarily to accelerate development, produce initial code structures, generate documentation drafts, and assist with debugging.

All generated code and documentation were reviewed, tested, and modified by the project authors prior to inclusion in the final system. The authors assume full responsibility for the correctness, functionality, and design decisions associated with the completed project.